

UNIFECAP

MATHEUS BONDEZAN

**Flash Store: Entrega contínua de uma
plataforma de pedidos em
microsserviços – Do Docker Compose ao
Kubernetes com observabilidade e
CI/CD**

1. Introdução

O presente relatório técnico fundamenta a proposta de modernização da infraestrutura da Flash Store, um e-commerce que, devido ao seu rápido crescimento, vem enfrentando instabilidades em produção, como indisponibilidades durante *deploys* e dificuldades para escalar em picos de acesso. Para solucionar esses gargalos e preparar o ambiente para a próxima campanha promocional, propõe-se a adoção de uma arquitetura *cloud-native* baseada em microsserviços, suportada por práticas modernas de Cloud DevOps. O objetivo é estabelecer um fluxo de entrega contínua, seguro e observável, do ambiente de desenvolvimento até a produção.

2. Arquitetura de Microsserviços e o Papel do DevOps em Ambientes Cloud-Native

A arquitetura de microsserviços substitui aplicações monolíticas por um conjunto de serviços pequenos, independentes e focados em capacidades de negócio específicas (como Pedidos, Pagamentos e Estoque). Cada serviço pode ser desenvolvido, implantado e escalado de forma independente. Em um ambiente *cloud-native*, essa abordagem maximiza a resiliência e a flexibilidade.

No entanto, gerenciar múltiplos serviços distribuídos aumenta consideravelmente a complexidade operacional. É aqui que o papel do DevOps se torna indispensável. DevOps não é apenas um conjunto de ferramentas, mas uma cultura que une desenvolvimento e operações para automatizar e monitorar todo o ciclo de vida do software. Neste contexto, o DevOps garante que os desenvolvedores tenham ambientes padronizados, enquanto a operação mantém a confiabilidade do sistema por meio de infraestrutura como código, testes automatizados e esteiras de *deploy* padronizadas, reduzindo o tempo de entrega e o risco de falhas humanas.

3. Containerização e Orquestração: Docker vs. Kubernetes

A containerização permite empacotar a aplicação e suas dependências em uma única unidade, garantindo que o software execute da mesma forma em qualquer ambiente.

- **Docker:** É a ferramenta ideal para criar, gerenciar e executar contêineres individualmente. No contexto da Loja Veloz, ele é utilizado para padronizar o ambiente de desenvolvimento local via Docker Compose, garantindo que o time de produto trabalhe em um cenário idêntico ao de teste.

- **Kubernetes (K8s):** Enquanto o Docker cuida do contêiner, o Kubernetes é o orquestrador que gerencia o ciclo de vida desses contêineres em escala de produção. Ele resolve problemas de disponibilidade ao reiniciar contêineres que falham e facilita a escalabilidade automática para suportar picos de tráfego, como o previsto para a campanha promocional da empresa.
- **Quando usar cada um:** O Docker é a escolha para o desenvolvimento e construção da imagem. O Kubernetes entra em cena quando a complexidade exige alta disponibilidade, balanceamento de carga e gerenciamento de múltiplos microsserviços em um cluster.

4. CI/CD em Ambientes Distribuídos

Para que a Flash Store recupere a agilidade sem comprometer a estabilidade, a implementação de uma esteira de *Continuous Integration* e *Continuous Deployment* (CI/CD) é obrigatória.

- **Continuous Integration (CI):** Foca na automação de builds e testes assim que o código é alterado. Isso evita que erros de integração cheguem às fases avançadas, garantindo a qualidade do código através de *linters* e testes unitários.
- **Continuous Deployment (CD):** Automatiza a entrega do artefato validado no ambiente de produção. Em ambientes de microsserviços, o CD permite aplicar estratégias de deploy de baixo risco, como o *Rolling Update* (atualização gradual) ou *Canary* (liberação para um pequeno grupo de usuários), resolvendo o problema de indisponibilidade durante os lançamentos de novas versões.

5. Conceitos de Observabilidade: Métricas, Logs e Traces

Em sistemas distribuídos complexos como o da Loja Veloz, a monitorização tradicional (saber se o servidor está "on" ou "off") é insuficiente. A observabilidade permite entender o estado interno do sistema a partir dos seus dados externos, utilizando três pilares fundamentais:

- **Métricas:** Dados quantitativos sobre o sistema ao longo do tempo (ex: uso de CPU, tempo de resposta da API de Pedidos, taxa de erros 5xx). São essenciais para disparar alertas e gatilhos de escalabilidade automática (HPA).
- **Logs:** Registros de eventos textuais e imutáveis que ocorrem em um serviço. No modelo *cloud-native*, os logs devem ser tratados como fluxos de eventos (*streams*), permitindo identificar exatamente o que aconteceu em um erro específico.
- **Traces (Rastreamento Distribuído):** Permitem acompanhar o ciclo de vida de uma requisição enquanto ela atravessa os diversos microsserviços (Gateway -> Pedidos ->

Estoque). Isso é vital para identificar em qual ponto da cadeia ocorre uma falha ou um gargalo de latência.

6. Justificativa das Decisões Arquiteturais

As escolhas técnicas para este projeto visam resolver os problemas críticos de indisponibilidade e falta de escala da Loja Veloz. A adoção do **Kubernetes** justifica-se pela necessidade de orquestração robusta, garantindo que o serviço de pagamentos e estoque se recuperem automaticamente de falhas. O uso de **Docker Compose** no desenvolvimento padroniza o ambiente para que o erro encontrado pelo desenvolvedor seja o mesmo do ambiente de QA. A implementação de uma esteira de **CI/CD** com estratégias de *Rolling Update* justifica-se pela eliminação do tempo de inatividade durante as atualizações, permitindo que a empresa realize múltiplos *deploys* por dia com segurança, mesmo durante a campanha promocional. Por fim, o esqueleto de **Terraform** garante que a infraestrutura seja replicável e auditável, evitando configurações manuais propensas a erro.